CRUD Data Karyawan

Service	Image	Port	Description
web	PHP 8.2 Apache (custom Dockerfile)	8080	CodeIgniter 3 application
database	mcr.microsoft.com/mssql/server:2022-latest	1433	Microsoft SQL Server 2022

Environments

File	Type	Description
`docker-compose.yml`	Production	No volume mounts, optimized runtime
`docker-compose.dev.yml`	Development	Source code mounted, hot reload enabled

🗄️ Database Schema

Tables

employees — Main employee data

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
nip	VARCHAR(20)	NOT NULL, UNIQUE
nama	VARCHAR(100)	NOT NULL
jenis_kelamin	VARCHAR(20)	NOT NULL, CHECK (jenis_kelamin IN ('Laki - Laki', 'Perempuan'))
jabatan_id	INT	NOT NULL, FK → jabatan(id)
tanggal_aktif_jabatan	DATE	NOT NULL
tanggal_masuk	DATE	NOT NULL
status_karyawan	VARCHAR(20)	NOT NULL, CHECK (status_karyawan IN ('Permanen', 'Kontrak'))
is_active	VARCHAR(10)	DEFAULT 'active'

users — Authentication

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
username	VARCHAR(50)	NOT NULL, UNIQUE
password	VARCHAR(255)	NOT NULL (hashed)

role	VARCHAR(20)	NOT NULL
is_active	VARCHAR(10)	DEFAULT 'active'

jabatan — Job position categories

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
nama_jabatan	VARCHAR(100)	NOT NULL, UNIQUE
deskripsi	VARCHAR(255)	NULL
is_active	VARCHAR(10)	DEFAULT 'active'

positions — Normalized position reference (for history tracking)

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
created_at	DATETIME	DEFAULT GETDATE()
updated_at	DATETIME	DEFAULT GETDATE()

employee_position_history — Position change log

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
employee_id	INT	NOT NULL, FK → employees(id)
position_id	INT	NOT NULL, FK → positions(id)
start_date	DATE	NOT NULL
end_date	DATE	NULL
created_at	DATETIME	DEFAULT GETDATE()

📁 Project Structure

```
├── docker/
│   ├── Dockerfile
│   ├── init-db.php             # DB init + positions + history seed
│   └── start.sh
├── application/
│   ├── config/
│   │   ├── autoload.php
│   │   ├── config.php
```

```
│   │   ├── database.php        # sqlsrv driver
│   │   └── routes.php          # All routes incl. positions & history
│   ├── controllers/
│   │   ├── Auth.php
│   │   ├── Dashboard.php       # Admin/User differentiated
│   │   ├── Employee.php        # CRUD + history API + position tracking
│   │   ├── Jabatan.php
│   │   └── Position.php        # Position CRUD
│   ├── models/
│   │   ├── Employee_model.php  # Gender stats, simple list
│   │   ├── User_model.php
│   │   ├── Jabatan_model.php
│   │   ├── Position_model.php
│   │   └── Position_history_model.php
│   └── views/
│       ├── layout/
│       │   ├── header.php      # Sidebar with Posisi & Riwayat menus
│       │   └── footer.php
│       ├── auth/login.php
│       ├── dashboard/index.php # Admin vs User dashboard
│       ├── employee/
│       │   ├── list.php        # History modal button
│       │   ├── form.php        # Position dropdown
│       │   └── history.php     # History timeline page
│       ├── jabatan/
│       │   ├── list.php
│       │   └── form.php
│       └── positions/
│           ├── list.php        # Position management
│           └── form.php
├── assets/
│   ├── css/
│   └── js/
├── docker-compose.yml
├── docker-compose.dev.yml
└── README.md
```

🌐 API Endpoints

Authentication

POST `/login`

- **Request (application/x-www-form-urlencoded):**
  - `username` (string, required)
  - `password` (string, required)
- **Response:**
  - Sukses:

```
{ "status": true, "message": "Login berhasil!", "redirect":
"http://localhost:8080/dashboard" }
```

- Gagal:

```
{ "status": false, "message": "Username atau password salah." }
```

GET `/logout`

- **Response:** Redirect ke halaman login, session dihapus.

Dashboard

Method	Endpoint	Description
GET	`/dashboard`	Dashboard page
GET	`/api/dashboard/stats`	Statistics (admin gets extra data)

Employee CRUD

GET `/api/employees/list`

- **Query Params:**
  - `limit` (int, optional, default: 10)
  - `offset` (int, optional, default: 0)
  - `search` (string, optional)
  - `sort_by` (string, optional, default: "id")
  - `sort_dir` (string, optional, default: "ASC")
- **Response:**

```
{
   "status": true,
   "data": [ { ...employee }, ... ],
   "total": 100,
   "limit": 10,
   "offset": 0
}
```

GET `/api/employees/get/{id}`

- **Response:**
  - Sukses:

```
{ "status": true, "data": { ...employee } }
```

  - Gagal:

```
{ "status": false, "message": "Karyawan tidak ditemukan." }
```

POST `/api/employees/create`

- **Request (application/x-www-form-urlencoded):**

    - `nip`, `nama`, `jenis_kelamin`, `jabatan`, `tanggal_aktif_jabatan`, `tanggal_masuk`, `status_karyawan`, `is_active`
- **Response:**
    - Sukses:

      ```
      { "status": true, "message": "Karyawan berhasil ditambahkan.", "id": 1
      }
      ```

    - Gagal validasi:

      ```
      { "status": false, "message": "Validasi gagal.", "errors": { "nip":
      "NIP wajib diisi" } }
      ```

    - NIP sudah digunakan:

      ```
      { "status": false, "message": "NIP sudah digunakan." }
      ```

POST `/api/employees/update/{id}`

- **Request:** Sama seperti create.
- **Response:** Mirip dengan create, dengan pesan `"Karyawan berhasil diperbarui."`

POST `/api/employees/delete/{id}`

- **Response:**
    - Sukses:

      ```
      { "status": true, "message": "Karyawan berhasil dihapus." }
      ```

    - Gagal:

      ```
      { "status": false, "message": "Gagal menghapus karyawan." }
      ```

GET `/api/employees/{id}/history`

- **Response:**

  ```
  {
     "status": true,
     "data": {
        "employee": { ...employee },
        "history": [ { ...position_history }, ... ]
     }
  }
  ```

Penjelasan Alur

- **Frontend**: Komponen utama (login, dashboard, list, form, history) melakukan request AJAX (GET/POST) ke endpoint backend.

- **Backend**: Controller menerima request, memproses validasi, otorisasi, dan business logic, lalu berinteraksi dengan model.
- **Model**: Model melakukan query ke database (SQL Server) untuk mengambil, menambah, mengubah, atau menghapus data.
- **Database**: Tabel utama: `users`, `employees`, `jabatan`, `positions`, `employee_position_history`.

Contoh alur:

- User login → POST `/login` → Auth Controller → User_model → `users`
- Lihat data karyawan → GET `/api/employees/list` → Employee Controller → Employee_model → `employees` JOIN `jabatan`
- Tambah karyawan → POST `/api/employees/create` → Employee Controller → Employee_model/Position_history_model → `employees`, `employee_position_history`

Data Rules

- Each employee may have multiple position history records
- First position is automatically logged when employee is created
- The **latest/current position** is the record where `end_date IS NULL`
- Position changes are tracked automatically — no manual history entry needed

---

👥 User Roles & Dashboard Differentiation

Admin Dashboard

Widget	Description
Karyawan Aktif	Count of active employees
Laki-laki	Male employee count
Perempuan	Female employee count
Permanen	Permanent employee count
Kontrak	Contract employee count
Total Jabatan	Total positions count
Perubahan Jabatan Terbaru	Latest 5 position changes
Quick Links	Positions, History pages

User Dashboard

Widget	Description
Karyawan Aktif	Count of active employees
Laki-laki	Male employee count
Perempuan	Female employee count
Permanen	Permanent employee count

Kontrak	Contract employee count

User dashboard does **NOT** contain: Total Jabatan, position management widgets, latest changes table, or position/history quick links.

Permissions

Action	Administrator	User
View employees	✅	✅
Add/Edit/Delete employees	✅	❌
View position history	✅	✅
Change positions	✅	❌
Manage positions	✅	❌

📊 Dummy Data

Positions (10 records)

Manager IT, Staff Keuangan, Staff HRD, Staff Marketing, Staff Administrasi, Supervisor HR, Asisten Manager Keuangan, Sekretaris, Manager Produksi, Admin Produksi

Employees (5 records)

NIP	Nama	Jabatan	Status
NIP001	Lukman Hakim	Manager IT	Tetap
NIP002	Saiful Anwar	Staff Keuangan	Tetap
NIP003	Sinta Mei	Staff HRD	Kontrak
NIP004	Tubagus	Staff Marketing	Tetap
NIP005	Nana M	Staff Administrasi	Kontrak

Users

Username	Password	Role
admin	admin123	administrator
user	user123	user

🚀 Quick Start

Production

```
docker-compose up -d
```

Development

```
docker-compose -f docker-compose.dev.yml up -d
```

Access

- **Application**: http://localhost:8080
- **Login**: admin / admin123

Stop & Reset

```
docker-compose down       # Stop
docker-compose down -v    # Stop + reset database
docker-compose up -d      # Start fresh
```

| password | VARCHAR(255) | NOT NULL (hashed) | | role | VARCHAR(20) | NOT NULL | | is_active | VARCHAR(10) | DEFAULT 'active' |

jabatan — Job position categories

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
nama_jabatan	VARCHAR(100)	NOT NULL, UNIQUE
deskripsi	VARCHAR(255)	NULL
is_active	VARCHAR(10)	DEFAULT 'active'

positions — Normalized position reference (for history tracking)

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
name	VARCHAR(100)	NOT NULL
created_at	DATETIME	DEFAULT GETDATE()
updated_at	DATETIME	DEFAULT GETDATE()

employee_position_history — Position change log

Column	Type	Constraints
id	INT IDENTITY	PRIMARY KEY
employee_id	INT	NOT NULL, FK → employees(id)

position_id	INT	NOT NULL, FK → positions(id)
start_date	DATE	NOT NULL
end_date	DATE	NULL
created_at	DATETIME	DEFAULT GETDATE()

📁 Project Structure

```
├── docker/
│   ├── Dockerfile
│   ├── init-db.php             # DB init + positions + history seed
│   └── start.sh
├── application/
│   ├── config/
│   │   ├── autoload.php
│   │   ├── config.php
│   │   ├── database.php        # sqlsrv driver
│   │   └── routes.php          # All routes incl. positions & history
│   ├── controllers/
│   │   ├── Auth.php
│   │   ├── Dashboard.php       # Admin/User differentiated
│   │   ├── Employee.php        # CRUD + history API + position tracking
│   │   ├── Jabatan.php
│   │   └── Position.php        # Position CRUD
│   ├── models/
│   │   ├── Employee_model.php  # Gender stats, simple list
│   │   ├── User_model.php
│   │   ├── Jabatan_model.php
│   │   ├── Position_model.php
│   │   └── Position_history_model.php
│   └── views/
│       ├── layout/
│       │   ├── header.php      # Sidebar with Posisi & Riwayat menus
│       │   └── footer.php
│       ├── auth/login.php
│       ├── dashboard/index.php # Admin vs User dashboard
│       ├── employee/
│       │   ├── list.php        # History modal button
│       │   ├── form.php        # Position dropdown
│       │   └── history.php     # History timeline page
│       ├── jabatan/
│       │   ├── list.php
│       │   └── form.php
│       └── positions/
│           ├── list.php        # Position management
│           └── form.php
├── assets/
│   ├── css/
│   └── js/
```

```
├── docker-compose.yml
├── docker-compose.dev.yml
└── README.md
```

🌐 API Endpoints

Authentication

POST `/login`

- **Request (application/x-www-form-urlencoded):**
  - `username` (string, required)
  - `password` (string, required)
- **Response:**
  - Sukses:

    ```
    { "status": true, "message": "Login berhasil!", "redirect":
    "http://localhost:8080/dashboard" }
    ```

  - Gagal:

    ```
    { "status": false, "message": "Username atau password salah." }
    ```

GET `/logout`

- **Response:** Redirect ke halaman login, session dihapus.

Dashboard

Method	Endpoint	Description
GET	`/dashboard`	Dashboard page
GET	`/api/dashboard/stats`	Statistics (admin gets extra data)

Employee CRUD

GET `/api/employees/list`

- **Query Params:**
  - `limit` (int, optional, default: 10)
  - `offset` (int, optional, default: 0)
  - `search` (string, optional)
  - `sort_by` (string, optional, default: "id")
  - `sort_dir` (string, optional, default: "ASC")
- **Response:**

```
{
   "status": true,
   "data": [ { ...employee }, ... ],
```

```
    "total": 100,
    "limit": 10,
    "offset": 0
}
```

GET `/api/employees/get/{id}`

- **Response:**
  - Sukses:

    ```
    { "status": true, "data": { ...employee } }
    ```

  - Gagal:

    ```
    { "status": false, "message": "Karyawan tidak ditemukan." }
    ```

POST `/api/employees/create`

- **Request (application/x-www-form-urlencoded):**
  - `nip`, `nama`, `jenis_kelamin`, `jabatan`, `tanggal_aktif_jabatan`, `tanggal_masuk`, `status_karyawan`, `is_active`
- **Response:**
  - Sukses:

    ```
    { "status": true, "message": "Karyawan berhasil ditambahkan.", "id": 1
    }
    ```

  - Gagal validasi:

    ```
    { "status": false, "message": "Validasi gagal.", "errors": { "nip":
    "NIP wajib diisi" } }
    ```

  - NIP sudah digunakan:

    ```
    { "status": false, "message": "NIP sudah digunakan." }
    ```

POST `/api/employees/update/{id}`

- **Request:** Sama seperti create.
- **Response:** Mirip dengan create, dengan pesan `"Karyawan berhasil diperbarui."`

POST `/api/employees/delete/{id}`

- **Response:**
  - Sukses:

    ```
    { "status": true, "message": "Karyawan berhasil dihapus." }
    ```

  - Gagal:

```
{ "status": false, "message": "Gagal menghapus karyawan." }
```

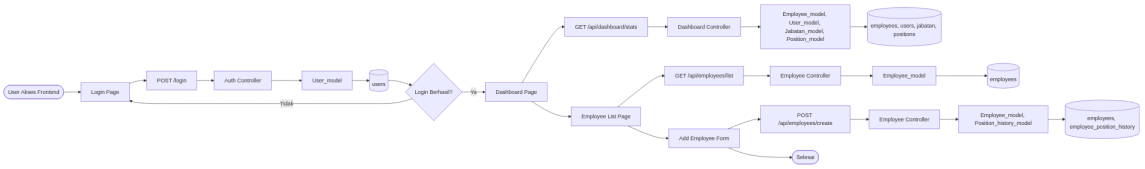
GET `/api/employees/{id}/history`

- **Response:**

```
{
   "status": true,
   "data": {
      "employee": { ...employee },
      "history": [ { ...position_history }, ... ]
   }
}
```

🔄 Flowchart Proses CRUD Data Karyawan

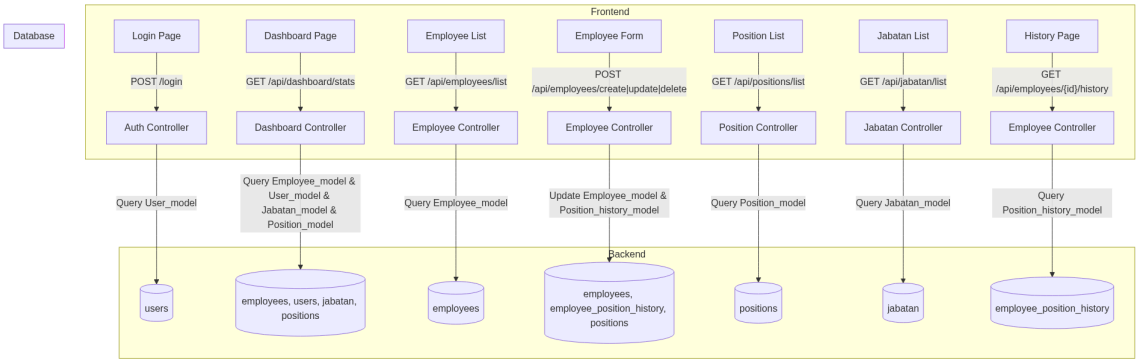
Flowchart berikut menggambarkan alur proses utama aplikasi dari frontend, backend, hingga database:



Aplikasi CRUD Data Karyawan menggunakan CodeIgniter 3, Microsoft SQL Server, dan Gentelella Admin Template, berjalan di Docker.

🖥️ End-to-End Architecture (Frontend–Backend–Database)

Overview



Penjelasan Alur

- **Frontend**: Komponen utama (login, dashboard, list, form, history) melakukan request AJAX (GET/POST) ke endpoint backend.

- **Backend**: Controller menerima request, memproses validasi, otorisasi, dan business logic, lalu berinteraksi dengan model.
- **Model**: Model melakukan query ke database (SQL Server) untuk mengambil, menambah, mengubah, atau menghapus data.
- **Database**: Tabel utama: `users`, `employees`, `jabatan`, `positions`, `employee_position_history`.

Contoh alur:

- User login → POST `/login` → Auth Controller → User_model → `users`
- Lihat data karyawan → GET `/api/employees/list` → Employee Controller → Employee_model → `employees` JOIN `jabatan`
- Tambah karyawan → POST `/api/employees/create` → Employee Controller → Employee_model/Position_history_model → `employees`, `employee_position_history`

Data Rules

- Each employee may have multiple position history records
- First position is automatically logged when employee is created
- The **latest/current position** is the record where `end_date IS NULL`
- Position changes are tracked automatically — no manual history entry needed

---

👥 User Roles & Dashboard Differentiation

Admin Dashboard

Widget	Description
Karyawan Aktif	Count of active employees
Laki-laki	Male employee count
Perempuan	Female employee count
Permanen	Permanent employee count
Kontrak	Contract employee count
Total Jabatan	Total positions count
Perubahan Jabatan Terbaru	Latest 5 position changes
Quick Links	Positions, History pages

User Dashboard

Widget	Description
Karyawan Aktif	Count of active employees
Laki-laki	Male employee count
Perempuan	Female employee count
Permanen	Permanent employee count

Kontrak	Contract employee count

User dashboard does **NOT** contain: Total Jabatan, position management widgets, latest changes table, or position/history quick links.

Permissions

Action	Administrator	User
View employees	✅	✅
Add/Edit/Delete employees	✅	❌
View position history	✅	✅
Change positions	✅	❌
Manage positions	✅	❌

📊 Dummy Data

Positions (10 records)

Manager IT, Staff Keuangan, Staff HRD, Staff Marketing, Staff Administrasi, Supervisor HR, Asisten Manager Keuangan, Sekretaris, Manager Produksi, Admin Produksi

Employees (5 records)

NIP	Nama	Jabatan	Status
NIP001	Lukman Hakim	Manager IT	Tetap
NIP002	Saiful Anwar	Staff Keuangan	Tetap
NIP003	Sinta Mei	Staff HRD	Kontrak
NIP004	Tubagus	Staff Marketing	Tetap
NIP005	Nana M	Staff Administrasi	Kontrak

Users

Username	Password	Role
admin	admin123	administrator
user	user123	user

🚀 Quick Start

Production

```
docker-compose up -d
```

Development

```
docker-compose -f docker-compose.dev.yml up -d
```

Access

- **Application**: http://localhost:8080
- **Login**: admin / admin123

Stop & Reset

```
docker-compose down       # Stop
docker-compose down -v    # Stop + reset database
docker-compose up -d      # Start fresh
```